



Documento del proyecto

Fundamentos de programación

Integrantes:

Daniel Alejandro Loza Ramírez 00105326

David Alejandro Morales Mejía 00148626

Angel Eduardo Miguel Vividor 00184026



Descripción general del proyecto

Este proyecto implica un juego de lucha por turnos, construido enteramente en C++ y diseñado para funcionar en la consola. Busca emular, de forma básica, las épicas batallas que encontramos en el universo Pokémon. La aplicación deja al usuario batallar contra un contrincante controlado por la IA, la computadora, después de elegir un luchador de entre siete posibles seres, todos ellos portando características únicas.

El sistema maneja los datos de cada ser vivo utilizando arreglos paralelos. En estos se guardan sus nombres, puntos de vida (HP) fundamentales, sus elementos (Fuego, Agua, Planta, Eléctrico, Tierra, Hielo y Normal), y, además, un trío de ataques para cada uno, donde figura el nombre del ataque y su respectivo daño. A mayores, cada uno de estos seres vivos tiene una representación visual, un "sprite" hecho en arte ASCII, el cual se muestra tanto al seleccionar al personaje como a lo largo de la pelea, añadiendo así una capa visual atractiva a la experiencia de la consola.

La jugabilidad se desarrolla mediante un sistema de turnos donde el jugador escoge uno de los movimientos disponibles que su personaje domina. El cálculo del daño infligido contempla varios aspectos que añaden dinamismo y rejugable al combate: hay un elemento azaroso que modifica el daño inicial, la probabilidad de un golpe crítico capaz de doblar el daño, y un entramado de ventajas y desventajas elementales capaces de acrecentar o mermar el daño en base a las relaciones de eficacia (el agua es superior al fuego por ejemplo o el fuego vence a la planta).

Siguiendo a la intervención del jugador, la CPU replica con una arremetida elegida al azar, aplicando la misma fórmula de cálculo.

En el transcurso de cada ronda se proporciona al usuario data clara de la situación del duelo a través de barras de vida graficas que muestran en tiempo real las unidades de vida faltantes de cada combatiente, los retratos de los luchadores presentados uno al lado del otro, y comunicaciones descriptivas de los actos llevados a cabo los embates críticos y la eficacia de las ofensivas.

La lid proseguirá hasta que la salud de uno de los contendientes caiga a cero, instante en el cual se declara al triunfador y la cantidad de vueltas que han transcurrido. El programa implementa también un módulo de validación de datos que protege la fortaleza del sistema permitiendo solamente entradas numeradas dentro de lo

establecido y rechazando vacíos no numéricos o fuera de rango. Además el juego proporciona la posibilidad de recomenzar el juego tantas veces como se necesite asegurando muchos juegos en una sesión.

En resumen, el proyecto ilustra la aplicación concreta de principios base de la programación estructurada como el uso de listas y matrices estructuras de repetición y decisión funciones independientes generación de números al azar y la validación de la información de entrada todo en una aplicación funcional y fácil de usar.

Objetivo del proyecto

Crear un juego interactivo de combate por turnos, Batalla Pokémon se llama, usando programación estructurada en C++, para que lo estudiado se pueda poner en práctica. El sistema hace que el usuario escoja un personaje de una lista y se enfrente a la máquina, escogiendo ataques de un menú, mientras el programa calcula cuanto daño se hace, cambia la vida de los dos y dice quién gana. Esto servirá para mostrar que el grupo puede entender un problema, planear una respuesta con un flujograma, y hacer un programa que funciona, bien organizado y fácil de entender, usando solo lo que vimos en clase: tipos de datos, operadores, condicionales, selecciones, bucles, arreglos y archivos.

Reglas o funcionamiento

El torneo de Combate Pokémon se desarrolla como una lucha mano a mano mediante turnos, entre un jugador y una unidad central de proceso (CPU) que controla al rival. Sus reglas de funcionamiento van en orden así:

1. La selección de los Pokemones. Cuando empieza el juego, la aplicación muestra un listado de siete pokemones disponibles, indicando su número, su nombre, sus puntos de vida (HP) y su tipo elemental Fuego Agua Planta Eléctrico Tierra Hielo o Normal. El jugador escoge su Pokemon metiendo un número del 1 al 7. Después, la maquina selecciona automáticamente un Pokemon distinto al del jugador.

2. Los atributos de los Pokemones. Cada Pokemon tiene un valor concreto de puntos de vida además de tres ataques únicos, cada uno con una puntuación de poder determinada que fija el daño básico que puede infligir.

3. El desarrollo por etapas. La batalla avanza por rondas. En cada ronda, primeramente, participa el jugador, que escoge uno de sus tres ataques de la lista; enseguida, la maquina responde, escogiendo un ataque sin ninguna lógica predeterminada. Los dos asaltos ocurren y se ve dentro del mismo periodo.

4. El cálculo del daño que se se realiza. El daño causado por cada golpe depende mucho de la fuerza del mismo ataque, sumándole un número al azar entre 0 y 4 que agrega sorpresa a cada encuentro. A este número se le hace las siguientes variaciones:

Un golpe tan fuerte que arrasa: una probabilidad de 1 de cada 10 hay, que un golpe sea crítico, duplicando el daño si esto pasa.

El tipo del Pokemon manda: si el tipo del atacante es superior al del otro, el golpe es buenísimo y el daño aumenta (multiplicado por 1.5). Los tipos superiores son: Fuego a Planta, Agua a Fuego, Planta a Agua, Rayo a Agua y Tierra a Rayo.

El tipo no ayuda nada: si, por el otro lado, el atacante es débil contra el oponente, el golpe no hace mucho y el daño baja (multiplicado por 0.75).

Siempre, el daño más bajo es 1.

5. El cambiar del estado. Después de cada ofensiva, la reserva vital del Pokemon impactado decrementa dependiendo de la lesión, pero nunca bajan de cero. Las rondas muestran los indicadores vitales de los Pokemones, sus avatares y notificaciones sobre el golpe, el daño infligido, y si hubo un impacto critico o una desventaja.

6. Criterio de triunfo. La refriega continua hasta que la vitalidad de uno cae. Si el rival sucumbe, ganas; si no, pierdes. El sistema dice quien gano y cuantos ciclos duró.

7. Verificación de datos. Durante el combate, el sistema revisa lo que dice el usuario. Solo acepta números dentro de rangos ya definidos, y no acepta vacíos, símbolos raros, o números mal puestos, pide otro número de nuevo hasta que el valor este bien.

8. Reanudación del desafío. Cuando termina una pelea, el programa le pregunta al usuario si quieren pelear otra vez. Si esto es cierto entonces la secuencia se reinicia con la selección del personaje eso sí la respuesta es no pues el programa acaba con una carta de despedida.

Menú de funciones

La dinámica de Batalla Pokémon se da en una secuencia de menús interactivos diseñados para orientar al usuario durante el desarrollo del juego. Las opciones son:

1. Selección de Personaje. Al ejecutarlo, surge un catálogo numerado presentando las siete entidades jugables disponibles, detallando el nombre, vitalidad y afinidad elemental de cada una. El jugador debe optar por una cifra del 1 al 7, determinando así su avatar en la contienda.

2. Menú de Ataques. En cada fase del enfrentamiento, se exhiben los tres movimientos ofensivos asociados al personaje del jugador, especificando su designación y daño que hacen. La elección, vía un número del 1 al 3, concreta la acción hostil dirigida al adversario.

3. Rejugar. Tras terminar un combate, se ofrece al jugador reanudar la partida. El jugador deberá introducir "s" (afirmativo) para volver a jugar desde la fase de elección de avatar, o "n" (negativo) para dar por terminada la operación del software.

Explicación del uso de los temas vistos en clase

A continuación, se explica cómo se aplican en el proyecto los contenidos estudiados durante el ciclo.

Tipos de datos. El programa utiliza diversos tipos de datos según la información que maneja. Enteros (int) para los puntos de vida el daño los turnos y las opciones del menú. Cadenas de texto (string) para los nombres de los Pokemones, los tipos elementales, los nombres de los ataques y los sprites. Valores lógicos como (bool) para determinar situaciones como un golpe critico o la ventaja de tipo. Caracteres (char) para leer la respuesta del usuario al preguntar si desea volver a jugar. Adicionalmente se emplean constantes (const) para definir valores fijos como el número total de Pokemones y la cantidad de ataques.

Arreglos. Se utilizan arreglos para almacenar de forma ordenada la información de los personajes, como sus nombres, los puntos de vida base (hp_base) y sus tipos. Estos arreglos permiten acceder a los datos de cada personaje mediante un índice. Evitando declarar variables independientes para cada uno.

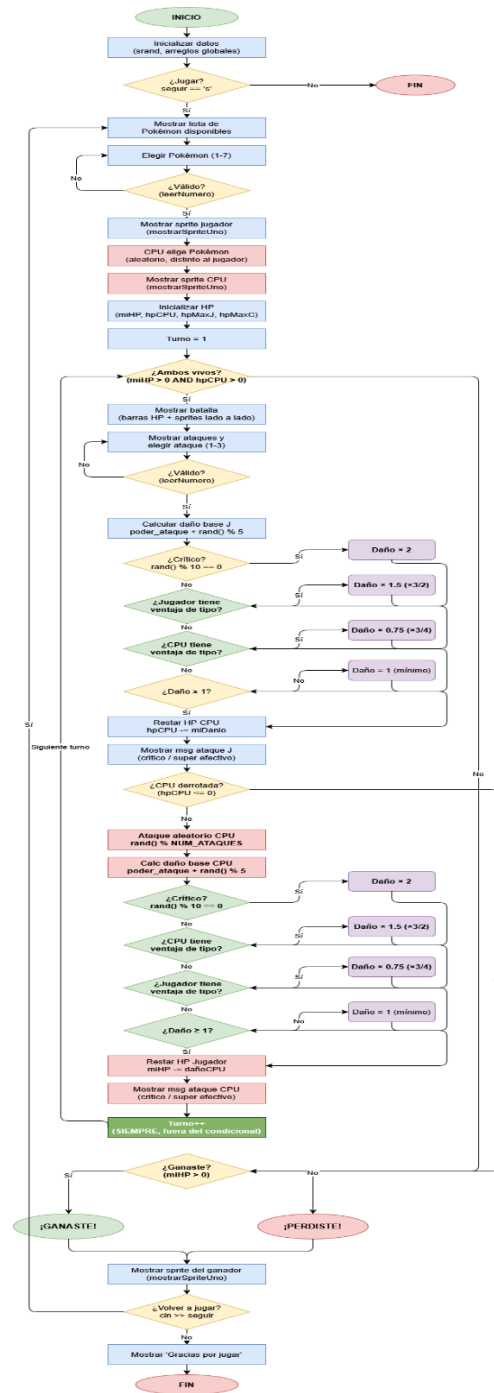
Matrices y arreglos. El proyecto emplea matrices para almacenar datos que dependen de dos dimensiones. Por ejemplo, las denominaciones de los ataques (nombre_ataque[]) y su potencia (poder_ataque[]) se conservan en arreglos cuya primera dimensión se refiere al peleador y la segunda a la secuencia de ataque. Asimismo, los gráficos en arte ASCII se archivan en una matriz de caracteres textuales, ordenados por luchador y por renglón.

Condicionales 'if'. Los 'if' se ejecutan en variados sitios de la mecánica del juego: para aclarar la superioridad o inferioridad entre categorías elementales, para comprobar si un golpe es letal, para garantizar que los perjuicios y las vitalidades no adquieran estados que no se desean y para determinar al vencedor al concluir el enfrentamiento.

Bucles. Se emplean bucles para repetir procedimientos de forma metódica. El bucle 'for' se utiliza para iterar sobre los arrays y matrices, al presentar la nómina de Pokemones, al plasmar los gráficos línea por línea o al edificar las representaciones de la vitalidad. El bucle 'while' gestiona el transcurso de la pelea (que se reitera en tanto los dos Pokemones posean energía), la confirmación de los datos (que se repite hasta obtener información correcta) y la opción de un nuevo encuentro.

Procedimientos. Mediante funciones, el programa se organiza; ellas dividen la lógica en bloques bien definidos y que se pueden usar de nuevo. Esto mejora el orden general, y la manera de entender el código. Ciertas funciones que se encuentran son: para visualizar los personajes (`mostrarSpriteUno`) y (`mostrarSpritesLadoALado`). Además, tiene una para la vida llamada (`mostrarBarraHP`). Para saber cuál es más fuerte entre tipos, se usa (`tieneVentaja`). Y para que los números del usuario estén bien, esta (`leerNumero`).

Flujograma digital



Explicación del manejo de archivos

El proyecto implementa el manejo de archivos como mecanismo de persistencia de datos, utilizando la librería `<fstream>` de C++. Se emplea específicamente la clase **ofstream (output file stream)**, que permite escribir información en archivos de texto plano.

Se definieron dos funciones dedicadas a esta tarea, ambas ubicadas en el archivo `archivos.cpp`:

guardarPartida(): crea o sobrescribe el archivo **partida_guardada.txt** cada vez que finaliza una batalla. Al abrirse en modo por defecto, el contenido anterior se reemplaza por completo, de modo que este archivo siempre refleja el resultado de la última partida jugada: nombres de los jugadores, Pokémon utilizados, tipo elemental, ganador y número de turnos.

guardarHistorial(): escribe en el archivo `historial_batallas.txt`, pero lo abre con el modificador **ios::app (append)**, lo que hace que cada nuevo registro se agregue al final del archivo sin borrar los anteriores. De esta forma se construye un historial acumulativo de todas las partidas jugadas durante la ejecución del programa.

Ambas funciones siguen la misma lógica de trabajo:

1. Abren el archivo correspondiente con **ofstream**.
2. Verifican con **is_open()** que la apertura fue exitosa; si no lo fue, se muestra un mensaje de error en consola en lugar de interrumpir el programa.
3. Escriben los datos formateados usando el operador `<<`.
4. Cierran el archivo con **close()**, liberando el recurso del sistema.

Adicionalmente, se utiliza la librería `<ctime>` para registrar la fecha y hora de cada partida: la función **time(0)** obtiene el instante actual del sistema y **ctime()** lo convierte a un formato de texto legible (por ejemplo, "Wed Jun 17 18:23:07 2026"), el cual se incluye en ambos archivos.

Este manejo de archivos permite que la información del juego persista incluso después de cerrar el programa, algo que no sería posible si los datos solo existieran en variables de memoria durante la ejecución.

Referencias o recursos de apoyo utilizados

Para el desarrollo del proyecto se consultaron los siguientes recursos de apoyo:

- Repositorio del docente Miguel Rivas, de la materia Fundamentos de Programación, disponible en: <https://github.com/Merivas/FUDP-2025>. En este repositorio el profesor publicó los códigos fuente y ejemplos desarrollados en clase, organizados por tema (arreglos, matrices, estructuras condicionales, funciones, structs, manejo de archivos, entre otros), los cuales sirvieron como referencia y punto de partida para varios de los conceptos aplicados en el proyecto.
- Claude, un asistente de inteligencia artificial desarrollado por Anthropic, el cual fue empleado para resolver dudas puntuales de sintaxis en C++, discutir buenas prácticas de organización modular del código y apoyar en la explicación de manejo de archivos.